

HTML · CSS · Bootstrap · JavaScript

Du niveau débutant à Expert

Cours complet, illustré & imprimable

 Débutant  Intermédiaire  Expert  CSS  Bootstrap



Table des matières

Chapitre	Contenu	Niveau
1. Bases du HTML	Structure, balises essentielles, liens, images, listes	 Débutant
2. HTML Intermédiaire	Sémantique, formulaires, tableaux, attributs HTML5	 Intermédiaire
3. HTML Expert	Accessibilité ARIA, SEO, méta Open Graph, Schema.org	 Expert
4. CSS — Style & Responsive	Boîte, Flexbox, Grid, variables, animations, media queries	 CSS
5. Bootstrap 5	Grille 12 col., utilitaires, composants (navbar, cards, modals)	 Bootstrap
6. Projet complet	Page portfolio HTML + CSS + Bootstrap de bout en bout	 Pratique
7. CSS pur vs Bootstrap	Comparatif, ressources et outils indispensables	 Pro
8. Feuille de route	Plan d'apprentissage pas-à-pas et checklist de mise en ligne	 Synthèse
9. JavaScript	Variables, conditions, boucles, fonctions, DOM, événements	 JS



1. Bases du HTML — Niveau Débutant

1.1 Qu'est-ce que le HTML ?

Le **HTML (HyperText Markup Language)** est le langage de base du web. Il structure le contenu d'une page : titres, paragraphes, images, liens, tableaux... Ce n'est pas un langage de programmation mais un **langage de balisage** — on « enveloppe » le contenu dans des balises pour lui donner du sens.



Analogie

HTML = la charpente d'une maison.

CSS = la peinture et la décoration.

JavaScript = l'électricité et la plomberie.

1.2 Structure minimale d'une page HTML

Toute page HTML valide doit respecter ce squelette :

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Ma première page</title>
  </head>
  <body>
    <h1>Bonjour le monde</h1>
    <p>Ceci est mon premier site.</p>
  </body>
</html>
```

Balise	Rôle
<!DOCTYPE html>	Indique au navigateur la version HTML (HTML5)
<html lang="fr">	Racine du document + langue pour l'accessibilité
<head>	Métadonnées : charset, titre, CSS, robots...
<meta charset="UTF-8">	Encodage universel (accents, symboles)
<meta name="viewport">	Rend la page responsive sur mobiles
<title>	Texte affiché dans l'onglet du navigateur
<body>	Contenu visible par l'utilisateur

1.3 Balises de texte essentielles

```
<h1>Titre principal (un seul par page)</h1>
<h2>Sous-titre</h2>
<h3>Sous-sous-titre</h3>    <!-- jusqu'à <h6> -->

<p>Un paragraphe de texte.</p>

<strong>Texte important (gras sémantique)</strong>
<em>Texte en italique (emphase)</em>
<mark>Texte surligné</mark>
<small>Petits caractères (mentions légales...)</small>
<del>Texte barré</del>
<ins>Texte ajouté</ins>
<code>Du code inline</code>
<br>    <!-- Saut de ligne -->
<hr>    <!-- Ligne horizontale de séparation -->
```

1.4 Liens et images

```
<!-- Lien externe -->
<a href="https://www.google.com" target="_blank" rel="noopener">Visiter
Google</a>

<!-- Lien interne -->
<a href="/contact.html">Contact</a>

<!-- Lien ancre (vers un id de la même page) -->
<a href="#section2">Aller à la section 2</a>

<!-- Image -->


<!-- Image avec lien -->
<a href="/galerie.html">
  
</a>
```

⚠ Règle d'or

L'attribut alt est OBLIGATOIRE sur toutes les images. Il est lu par les lecteurs d'écran (accessibilité) et indexé par Google (SEO).

1.5 Listes

```
<!-- Liste non ordonnée (puces) -->
<ul>
  <li>HTML</li>
  <li>CSS</li>
```

```
<li>JavaScript</li>
</ul>

<!-- Liste ordonnée (numérotée) -->
<ol>
  <li>Ouvrir l'éditeur</li>
  <li>Écrire le code</li>
  <li>Sauvegarder</li>
  <li>Ouvrir dans le navigateur</li>
</ol>

<!-- Liste de définitions -->
<dl>
  <dt>HTML</dt>
  <dd>Langage de structure du web</dd>
  <dt>CSS</dt>
  <dd>Langage de mise en forme</dd>
</dl>
```



2. HTML Intermédiaire — Sémantique & Formulaires

2.1 HTML sémantique

Les balises sémantiques donnent du **sens** au contenu (pas seulement du style). Elles aident les moteurs de recherche et les technologies d'assistance.

```
<header>
  <nav>
    <a href="/">Accueil</a>
    <a href="/blog">Blog</a>
  </nav>
</header>

<main>
  <section>
    <h2>Nos articles</h2>
    <article>
      <h3>Article 1</h3>
      <p>Contenu de l'article...</p>
    </article>
  </section>
  <aside>
    <h3>À lire aussi</h3>
  </aside>
</main>

<footer>
  <p>© 2026 Mon Site</p>
</footer>
```

2.2 Tableaux HTML

```
<table>
  <caption>Résultats du trimestre</caption>
  <thead>
    <tr>
      <th scope="col">Mois</th>
      <th scope="col">Ventes</th>
      <th scope="col">Croissance</th>
    </tr>
  </thead>
  <tbody>
    <tr><td>Janvier</td><td>1 200 €</td><td>+5%</td></tr>
    <tr><td>Février</td><td>1 450 €</td><td>+20%</td></tr>
  </tbody>
  <tfoot>
    <tr><td>Total</td><td colspan="2">2 650 €</td></tr>
  </tfoot>
</table>
```

⚠ Attention

Les tableaux HTML sont réservés aux données tabulaires (statistiques, calendriers, comparatifs). Ne jamais les utiliser pour la mise en page : utilisez CSS Flexbox ou Grid.

2.3 Formulaires HTML

Les formulaires permettent la saisie et l'envoi de données (inscription, contact, recherche...).

```
<form action="/traitement.php" method="POST" novalidate>

  <label for="nom">Nom complet *</label>
  <input type="text" id="nom" name="nom" required placeholder="Jean Dupont">

  <label for="email">Adresse email *</label>
  <input type="email" id="email" name="email" required>

  <label for="mdp">Mot de passe</label>
  <input type="password" id="mdp" name="mdp" minlength="8">

  <label for="age">Âge</label>
  <input type="number" id="age" name="age" min="18" max="120">

  <label for="pays">Pays</label>
  <select id="pays" name="pays">
    <option value="">-- Choisir --</option>
    <option value="fr">France</option>
    <option value="be">Belgique</option>
    <option value="ch">Suisse</option>
  </select>

  <fieldset>
    <legend>Genre</legend>
    <input type="radio" id="h" name="genre" value="h">
    <label for="h">Homme</label>
    <input type="radio" id="f" name="genre" value="f">
    <label for="f">Femme</label>
  </fieldset>

  <input type="checkbox" id="cgu" name="cgu" required>
  <label for="cgu">J'accepte les conditions générales</label>

  <label for="message">Message</label>
  <textarea id="message" name="message" rows="5"></textarea>

  <button type="submit">Envoyer</button>
  <button type="reset">Réinitialiser</button>
</form>
```

Types d'inputs utiles

type="..."	Exemple	Usage
------------	---------	-------

text	Jean Dupont	Texte libre
email	jean@site.com	Email validé automatiquement
password	••••••	Masqué
number	42	Valeur numérique
date	2026-01-01	Calendrier
tel	+32 497 000 000	Téléphone
url	https://...	Adresse web
range	0 → 100	Slider
color	#FF5733	Sélecteur couleur
file	document.pdf	Upload de fichier
hidden	token=abc	Donnée cachée
search	recherche...	Barre de recherche

● 3. HTML Expert — Accessibilité, SEO & Bonnes pratiques

3.1 Accessibilité (ARIA)

L'accessibilité garantit que le site est utilisable par tous, y compris les personnes en situation de handicap (malvoyants, motricité réduite, cognitif...).

```
<!-- Rôles ARIA -->
<nav aria-label="Navigation principale">
  <ul>
    <li><a href="/" aria-current="page">Accueil</a></li>
  </ul>
</nav>

<!-- Bouton personnalisé accessible -->
<div role="button" tabindex="0"
  aria-label="Fermer la fenêtre"
  onkeydown="if(event.key==='Enter'){fermer()}">x</div>

<!-- Image décorative (ignorée par les lecteurs d'écran) -->


<!-- Formulaire accessible -->
<div role="alert" aria-live="polite" id="erreur"></div>
<input aria-describedby="aide-email" aria-required="true">
<span id="aide-email">Format : nom@domaine.com</span>

<!-- Skip link (accessibilité clavier) -->
<a href="#contenu-principal" class="skip-link">
  Aller au contenu principal
</a>
```

3.2 Balises meta pour le SEO

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <!-- SEO de base -->
  <title>Titre de la page (50-60 caractères) | Nom du site</title>
  <meta name="description" content="Description de 150-160 caractères.">
  <meta name="robots" content="index, follow">
  <link rel="canonical" href="https://monsite.com/ma-page">

  <!-- Open Graph (réseaux sociaux) -->
  <meta property="og:title" content="Mon super article">
  <meta property="og:description" content="Résumé de l'article...">
  <meta property="og:image" content="https://monsite.com/images/og.jpg">
  <meta property="og:url" content="https://monsite.com/article">
  <meta property="og:type" content="website">
```

```

<!-- Twitter Cards -->
<meta name="twitter:card" content="summary_large_image">
<meta name="twitter:title" content="Mon super article">
<meta name="twitter:image" content="https://monsite.com/images/twitter.jpg">

<!-- Favicon -->
<link rel="icon" type="image/png" href="/favicon.png">
<link rel="apple-touch-icon" href="/apple-touch-icon.png">
</head>

```

3.3 Données structurées (Schema.org)

Les données structurées aident Google à comprendre et afficher votre contenu dans les résultats enrichis.

```

<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Article",
  "headline": "Apprendre le HTML en 2026",
  "author": { "@type": "Person", "name": "Jean Martin" },
  "datePublished": "2026-01-15",
  "image": "https://monsite.com/images/article.jpg",
  "publisher": {
    "@type": "Organization",
    "name": "Mon Site",
    "logo": { "@type": "ImageObject", "url": "https://monsite.com/logo.png" }
  }
}
</script>

```

3.4 Checklist expert HTML

- ✓ Une seule balise <main> par page
- ✓ Hiérarchie des titres respectée (h1 → h2 → h3...)
- ✓ Tous les attributs alt renseignés
- ✓ Formulaire avec label associé à chaque input
- ✓ Attribut lang sur <html>
- ✓ Viewport meta présent pour le responsive
- ✓ charset UTF-8 déclaré
- ✓ ARIA utilisé pour les composants interactifs
- ✓ HTML validé via <https://validator.w3.org>
- ✓ Pas de tableaux pour la mise en page



4. CSS — Du style de base au Responsive

4.1 Les trois façons d'écrire du CSS

```
<!-- 1. CSS externe (RECOMMANDÉ) -->
<link rel="stylesheet" href="style.css">

<!-- 2. CSS interne (dans <head>) -->
<style>
  h1 { color: red; }
</style>

<!-- 3. CSS inline (à éviter) -->
<p style="color: red; font-weight: bold;">Texte rouge</p>
```

4.2 Sélecteurs CSS

Sélecteur	Syntaxe	Cible
Élément	p { }	Tous les <p>
Classe	.btn { }	Éléments avec class="btn"
Identifiant	#header { }	L'élément avec id="header"
Universel	* { }	Tous les éléments
Descendant	div p { }	<p> à l'intérieur d'un <div>
Enfant direct	ul > li { }	enfant direct de
Adjacent	h2 + p { }	<p> juste après <h2>
Attribut	input[type="email"]	Inputs de type email
:hover	a:hover { }	Lien survolé
:focus	input:focus { }	Input actif (clavier)
:first-child	li:first-child	Premier élément de liste
:nth-child	tr:nth-child(2n)	Lignes paires
::before	p::before { }	Pseudo-élément avant le contenu

4.3 Le modèle de boîte (Box Model)

Chaque élément HTML est une boîte rectangulaire composée de 4 couches :

```
.ma-boite {
  width: 300px;
  height: 150px;
```

```

/* Padding : espace intérieur */
padding: 20px; /* tout autour */
padding: 10px 20px; /* haut/bas | gauche/droite */
padding: 5px 10px 15px 20px; /* haut | droite | bas | gauche */

border: 2px solid #333;
border-radius: 8px;

/* Margin : espace extérieur */
margin: 20px auto; /* centrer horizontalement */

box-sizing: border-box;
}

```



Astuce universelle

Ajoutez `* { box-sizing: border-box; }` en tout début de votre CSS. Cela simplifie le calcul des dimensions et évite de nombreux bugs d'affichage.

4.4 CSS Flexbox

Flexbox est le système de mise en page 1D (une ligne ou une colonne). Idéal pour aligner des éléments dans une direction.

```

.container {
  display: flex;
  flex-direction: row; /* row | column | row-reverse | column-reverse */
  justify-content: center; /* alignement sur l'axe principal */
  align-items: center; /* alignement sur l'axe croisé */
  flex-wrap: wrap; /* retour à la ligne si débordement */
  gap: 16px;
}

.enfant {
  flex: 1; /* prend l'espace disponible */
  flex-grow: 2; /* grandit 2× plus */
  flex-shrink: 0; /* ne rétrécit pas */
  flex-basis: 200px; /* taille de base */
  align-self: flex-start;
  order: 2;
}

/* Exemples pratiques */
nav { display: flex; justify-content: space-between; align-items: center; }
.hero { display: flex; justify-content: center; align-items: center; min-height: 100vh; }
.cards { display: flex; gap: 20px; flex-wrap: wrap; }
.card { flex: 1 1 250px; }

```

4.5 CSS Grid

CSS Grid est le système de mise en page 2D (lignes ET colonnes). Idéal pour les mises en page complexes.

```
.grille {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr; /* 3 colonnes */
  gap: 20px;
}

/* Zones nommées (mise en page complète) */
.layout {
  display: grid;
  grid-template-columns: 250px 1fr;
  grid-template-rows: 80px 1fr 60px;
  grid-template-areas:
    "header header"
    "sidebar contenu"
    "footer footer";
  min-height: 100vh;
}
header { grid-area: header; }
.sidebar { grid-area: sidebar; }
main { grid-area: contenu; }
footer { grid-area: footer; }

/* Grid responsive automatique */
.galerie {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  gap: 16px;
}

/* auto-fill + minmax = responsive SANS media query ! */
```

4.6 Variables CSS (Custom Properties)

```
:root {
  --couleur-primaire: #2874A6;
  --couleur-secondaire: #F39C12;
  --couleur-texte: #1C2833;
  --couleur-fond: #F2F3F4;
  --bordure-radius: 8px;
  --ombre: 0 2px 8px rgba(0,0,0,0.15);
  --transition: all 0.3s ease;
  --police: 'Segoe UI', Arial, sans-serif;
}

body {
  font-family: var(--police);
  color: var(--couleur-texte);
  background: var(--couleur-fond);
}
```

```

/* Mode sombre avec les mêmes variables */
@media (prefers-color-scheme: dark) {
  :root {
    --couleur-texte: #ECF0F1;
    --couleur-fond: #1C2833;
  }
}

```

4.7 Animations CSS

```

/* Transition (entre 2 états) */
.btn {
  background: #2874A6;
  transition: background 0.3s ease, transform 0.2s ease;
}
.btn:hover {
  background: #1A5276;
  transform: translateY(-2px);
}

/* Animation keyframe */
@keyframes apparition {
  from { opacity: 0; transform: translateY(30px); }
  to   { opacity: 1; transform: translateY(0); }
}
.carte { animation: apparition 0.6s ease forwards; }

/* Spinner infini */
@keyframes rotation {
  from { transform: rotate(0deg); }
  to   { transform: rotate(360deg); }
}
.spinner {
  width: 40px; height: 40px;
  border: 4px solid #ccc;
  border-top-color: #2874A6;
  border-radius: 50%;
  animation: rotation 0.8s linear infinite;
}

```

4.8 CSS Responsive & Media Queries

```

/* Approche Mobile First (recommandée) */

/* Base : mobile (< 576px) */
.container { padding: 16px; width: 100%; }
h1 { font-size: 1.8rem; }

/* Tablettes (>= 576px) */
@media (min-width: 576px) {
  .container { max-width: 540px; margin: 0 auto; }
}

```

```
h1 { font-size: 2.2rem; }
}

/* Laptops (>= 768px) */
@media (min-width: 768px) {
  .container { max-width: 720px; }
  .grid-2 { display: grid; grid-template-columns: 1fr 1fr; }
}

/* Desktops (>= 992px) */
@media (min-width: 992px) {
  .container { max-width: 960px; }
  .grid-3 { grid-template-columns: repeat(3, 1fr); }
}

/* Grands écrans (>= 1200px) */
@media (min-width: 1200px) {
  .container { max-width: 1140px; }
}

/* Media queries spéciales */
@media print { .sidebar, nav { display: none; } }
@media (prefers-reduced-motion: reduce) { * { animation: none !important; } }
@media (orientation: landscape) { .hero { min-height: 80vh; } }
```



5. Bootstrap 5 — Framework CSS professionnel

5.1 Intégrer Bootstrap 5 (CDN)

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mon projet Bootstrap</title>
  <!-- Bootstrap CSS -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
  <!-- Votre contenu ici -->

  <!-- Bootstrap JS (pour modals, dropdowns, etc.) -->
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
>
  </script>
</body>
</html>
```

5.2 La grille Bootstrap (12 colonnes)

Bootstrap divise l'écran en 12 colonnes. On indique combien de colonnes chaque élément doit occuper selon la taille d'écran.

```
<div class="container">
  <div class="row">
    <!-- 12/12 mobile, 8/12 md, 9/12 lg -->
    <div class="col-12 col-md-8 col-lg-9">Contenu principal</div>
    <!-- 12/12 mobile, 4/12 md, 3/12 lg -->
    <div class="col-12 col-md-4 col-lg-3">Sidebar</div>
  </div>

  <!-- Colonnes égales automatiques -->
  <div class="row">
    <div class="col">Colonne 1</div>
    <div class="col">Colonne 2</div>
    <div class="col">Colonne 3</div>
  </div>
</div>
```

Classe	< 576px	>= 576px	>= 768px	>= 992px	>= 1200px
col-*	Actif	—	—	—	—

col-sm-*	✓	✓	—	—	—
col-md-*	✓	✓	✓	—	—
col-lg-*	✓	✓	✓	✓	—
col-xl-*	✓	✓	✓	✓	✓

5.3 Classes utilitaires Bootstrap

```

<!-- Espacement (m=margin, p=padding, t=top, b=bottom, x=horizontal, y=vertical)
-->
<!-- Échelle : 0(0px) 1(4px) 2(8px) 3(16px) 4(24px) 5(48px) -->
<div class="mt-3 mb-4 px-2 py-5">...</div>

<!-- Affichage responsive -->
<div class="d-none d-md-block">Visible sur md+ seulement</div>
<div class="d-block d-md-none">Visible sur mobile seulement</div>

<!-- Flexbox -->
<div class="d-flex justify-content-between align-items-center gap-3">
  <span>Gauche</span><span>Droite</span>
</div>

<!-- Typographie -->
<p class="text-center text-muted fw-bold fs-5">Centré, gris, gras</p>
<h2 class="display-4 text-primary">Grand titre Bootstrap</h2>

<!-- Couleurs -->
<p class="text-success bg-light p-3 rounded">Succès vert sur fond clair</p>
<div class="bg-danger text-white p-2">Erreur</div>

<!-- Bordures & ombres -->
<div class="border border-primary rounded-3 shadow-sm p-3">Carte simple</div>

```

5.4 Composants Bootstrap essentiels

Navigation (Navbar)

```

<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand fw-bold" href="#">Mon Site</a>
    <button class="navbar-toggler" type="button"
      data-bs-toggle="collapse" data-bs-target="#menu">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="menu">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link active"
href="/">Accueil</a></li>
        <li class="nav-item"><a class="nav-link" href="/blog">Blog</a></li>
        <li class="nav-item">

```

```

        <a class="btn btn-primary ms-2" href="/contact">Contact</a>
    </li>
</ul>
</div>
</div>
</nav>

```

Cards (cartes)

```

<div class="row row-cols-1 row-cols-md-3 g-4">
  <div class="col">
    <div class="card h-100 shadow-sm">
      
      <div class="card-body">
        <h5 class="card-title">Titre de l'article</h5>
        <p class="card-text text-muted">Résumé de l'article...</p>
      </div>
      <div class="card-footer d-flex justify-content-between align-items-center">
        <small class="text-muted">15 jan 2026</small>
        <a href="#" class="btn btn-sm btn-outline-primary">Lire</a>
      </div>
    </div>
  </div>
</div>

```

Modal (fenêtre popup)

```

<!-- Bouton déclencheur -->
<button type="button" class="btn btn-primary"
  data-bs-toggle="modal" data-bs-target="#maModal">
  Ouvrir la fenêtre
</button>

<!-- Modal -->
<div class="modal fade" id="maModal" tabindex="-1"
  aria-labelledby="titreModal" aria-hidden="true">
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title" id="titreModal">Confirmation</h5>
        <button class="btn-close" data-bs-dismiss="modal" aria-label="Fermer"></button>
      </div>
      <div class="modal-body">Êtes-vous sûr de vouloir continuer ?</div>
      <div class="modal-footer">
        <button class="btn btn-secondary" data-bs-dismiss="modal">Annuler</button>
        <button class="btn btn-danger">Confirmer</button>
      </div>
    </div>
  </div>
</div>

```

Alertes & Badges

```
<!-- Alertes -->
<div class="alert alert-success alert-dismissible fade show" role="alert">
  Formulaire envoyé avec succès !
  <button class="btn-close" data-bs-dismiss="alert"></button>
</div>
<div class="alert alert-danger">Une erreur s'est produite.</div>
<div class="alert alert-warning">Attention, cette action est irréversible.</div>
<div class="alert alert-info">Nouvelle fonctionnalité disponible.</div>

<!-- Badges -->
<span class="badge bg-primary">Nouveau</span>
<span class="badge bg-danger rounded-pill">3 notifications</span>

<!-- Boutons -->
<button class="btn btn-primary">Principal</button>
<button class="btn btn-secondary">Secondaire</button>
<button class="btn btn-success">Succès</button>
<button class="btn btn-danger">Danger</button>
<button class="btn btn-outline-primary">Outline</button>
<button class="btn btn-lg btn-primary">Grand</button>
<button class="btn btn-sm btn-danger">Petit</button>
```

Formulaire Bootstrap

```
<form>
  <div class="mb-3">
    <label for="nom" class="form-label fw-semibold">Nom complet</label>
    <input type="text" class="form-control" id="nom" placeholder="Jean Dupont">
    <div class="form-text">Votre nom tel qu'il apparaîtra sur le site.</div>
  </div>

  <div class="mb-3">
    <label for="email" class="form-label fw-semibold">Email</label>
    <input type="email" class="form-control is-invalid" id="email">
    <div class="invalid-feedback">Veuillez entrer un email valide.</div>
  </div>

  <!-- Groupe d'inputs côte à côte -->
  <div class="input-group mb-3">
    <span class="input-group-text">€</span>
    <input type="number" class="form-control" placeholder="Montant">
    <span class="input-group-text">.00</span>
  </div>

  <div class="form-check mb-3">
    <input class="form-check-input" type="checkbox" id="cgu">
    <label class="form-check-label" for="cgu">J'accepte les CGU</label>
  </div>

  <button type="submit" class="btn btn-primary w-100">Envoyer</button>
</form>
```




6. Projet complet — Page de présentation Bootstrap

Page complète combinant HTML sémantique, CSS personnalisé et Bootstrap 5 :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Portfolio | Jean Martin</title>
  <meta name="description" content="Portfolio de Jean Martin, développeur web.">
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    :root { --primaire: #2874A6; --accent: #F39C12; }
    body { font-family: 'Segoe UI', Arial, sans-serif; }
    .hero {
      background: linear-gradient(135deg, #1C2833 0%, #2874A6 100%);
      min-height: 100vh;
      display: flex;
      align-items: center;
    }
    .carte-projet { transition: transform .3s ease, box-shadow .3s ease; }
    .carte-projet:hover { transform: translateY(-6px);
      box-shadow: 0 12px 24px rgba(0,0,0,.15); }
    @media (max-width: 576px) { .hero h1 { font-size: 2rem; } }
  </style>
</head>
<body>

  <!-- NAVIGATION -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark sticky-top">
    <div class="container">
      <a class="navbar-brand fw-bold" href="#">JM</a>
      <button class="navbar-toggler" data-bs-toggle="collapse" data-bs-
target="#nav">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="nav">
        <ul class="navbar-nav ms-auto gap-2">
          <li class="nav-item"><a class="nav-link"
href="#projets">Projets</a></li>
          <li class="nav-item"><a class="nav-link"
href="#competences">Compétences</a></li>
          <li class="nav-item">
            <a class="btn btn-warning btn-sm" href="#contact">Contact</a>
          </li>
        </ul>
      </div>
    </div>
  </nav>

  <!-- HERO -->
  <section class="hero text-white" id="accueil">
```

```

<div class="container text-center">
  <p class="text-warning fw-bold mb-2">Développeur Web Full-Stack</p>
  <h1 class="display-3 fw-bold mb-4">Jean Martin</h1>
  <p class="lead mb-5 text-white-50">
    Je crée des applications web modernes, rapides et accessibles.
  </p>
  <a href="#projets" class="btn btn-warning btn-lg me-3">Voir mes
projets</a>
  <a href="#contact" class="btn btn-outline-light btn-lg">Me contacter</a>
</div>
</section>

<!-- PROJETS -->
<section class="py-5 bg-light" id="projets">
  <div class="container">
    <h2 class="text-center mb-2">Mes Projets</h2>
    <p class="text-center text-muted mb-5">Sélection de mes réalisations
récentes</p>
    <div class="row row-cols-1 row-cols-md-2 row-cols-lg-3 g-4">
      <div class="col">
        <div class="card h-100 border-0 shadow-sm carte-projet">
          <div class="card-body">
            <span class="badge bg-primary mb-2">React</span>
            <h5 class="card-title">Application e-commerce</h5>
            <p class="card-text text-muted">
              Boutique en ligne avec panier et paiement Stripe.
            </p>
          </div>
          <div class="card-footer bg-transparent border-0">
            <a href="#" class="btn btn-outline-primary btn-sm">Voir →</a>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>

<!-- CONTACT -->
<section class="py-5 bg-dark text-white" id="contact">
  <div class="container" style="max-width:600px">
    <h2 class="text-center mb-4">Me contacter</h2>
    <form>
      <div class="mb-3">
        <input type="text" class="form-control bg-secondary border-0 text-
white"
          placeholder="Votre nom">
      </div>
      <div class="mb-3">
        <input type="email" class="form-control bg-secondary border-0 text-
white"
          placeholder="Email">
      </div>
      <div class="mb-3">
        <textarea class="form-control bg-secondary border-0 text-white"
          rows="4" placeholder="Votre message..."></textarea>
      </div>
      <button type="submit" class="btn btn-warning w-100">

```

```
fw-bold">Envoyer</button>
    </form>
  </div>
</section>

<footer class="text-center py-3 bg-black text-white-50">
  <small>© 2026 Jean Martin – Tous droits réservés</small>
</footer>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
  </script>
</body>
</html>
```



7. CSS pur vs Bootstrap — Quand choisir quoi ?

 CSS Pur	 Bootstrap
Apprentissage et compréhension	Prototypes rapides
Design sur mesure unique	Projets d'équipe avec conventions
Petits projets sans dépendances	Projets professionnels
Contrôle total sur le CSS	Gain de temps important
Performance maximale (0 Ko de lib)	Composants prêts à l'emploi
Animations et effets complexes	Grille responsive clé en main

Conseil de pro

Un bon développeur maîtrise les deux. Commencez par le CSS pur pour comprendre les fondations, puis adoptez Bootstrap pour gagner en productivité. En entreprise, les deux coexistent souvent dans le même projet.

7.1 Outils et ressources indispensables

- MDN Web Docs — <https://developer.mozilla.org> : la référence officielle
- W3C Validator — <https://validator.w3.org> : valider votre HTML
- Can I use — <https://caniuse.com> : compatibilité des fonctionnalités CSS
- CSS Tricks — <https://css-tricks.com> : guides Flexbox et Grid
- Bootstrap Icons — <https://icons.getbootstrap.com> : icônes gratuites
- Colors — <https://colors.co> : générateur de palettes de couleurs
- Google Fonts — <https://fonts.google.com> : polices web gratuites
- Figma — <https://figma.com> : maquettage UI/UX (version gratuite)
- VS Code — <https://code.visualstudio.com> : éditeur recommandé
- Extensions VS Code : Live Server (rechargement auto), Prettier (formatage), HTML CSS Support (auto-complétion)



8. Résumé — Votre feuille de route complète

Étape	Ce que vous apprenez	Résultat attendu
🟢 Étape 1	Structure HTML, balises de base, liens, images, listes	Page statique simple et valide
🟡 Étape 2	HTML sémantique, formulaires, tableaux, attributs	Pages organisées et structurées
🔴 Étape 3	Accessibilité ARIA, SEO, données structurées	Site professionnel et indexable
🎨 Étape 4	CSS sélecteurs, box model, couleurs, typographie	Mise en forme maîtrisée
📐 Étape 5	Flexbox, Grid, variables CSS, animations	Layouts modernes et dynamiques
📱 Étape 6	Media queries, mobile-first, responsive design	Site adaptatif sur tous écrans
🚀 Étape 7	Bootstrap : grille, composants, utilities	Projets rapides et professionnels
🖨️ Étape 8	Projet complet HTML+CSS+Bootstrap	Portfolio ou landing page
🟡 Étape 9	JavaScript : variables, conditions, boucles, fonctions	Pages interactives et dynamiques
🟡 Étape 10	DOM, événements, mini-projets JavaScript	Interactivité complète maîtrisée

Checklist finale avant mise en ligne

- ✓ HTML validé (validator.w3.org)
- ✓ Images optimisées et attributs alt renseignés
- ✓ Site testé sur Chrome, Firefox et Safari
- ✓ Site testé sur mobile (DevTools > mode responsive)
- ✓ Vitesse de chargement vérifiée (PageSpeed Insights)
- ✓ Contrastes de couleurs suffisants (WCAG AA minimum)
- ✓ Balises meta SEO complètes
- ✓ Formulaires fonctionnels et validés
- ✓ Liens testés (aucun lien cassé)
- ✓ HTTPS activé sur le serveur

Bonne pratique et bon code ! 🚀



9. JavaScript — L'interactivité et la logique

9.1 Qu'est-ce que JavaScript ?

JavaScript (JS) est le langage qui permet de rendre une page web interactive et dynamique.

- HTML = La structure (les murs).
- CSS = L'apparence (la peinture).
- JavaScript = Le comportement (l'électricité et la plomberie).

9.2 Les trois façons d'intégrer du JavaScript

Méthode	Syntaxe	Usage
Fichier externe (Recommandé)	<code><script src="app.js" defer></script></code>	Code propre, séparé et réutilisable. L'attribut defer charge le script après le HTML.
Interne (dans la page)	<code><script>console.log("Bonjour");</script></code>	Utile pour des tests rapides ou de très petits scripts.
Inline (À éviter)	<code><button onclick="alert('Clic !')"></code>	Mélange structure et logique. Difficile à maintenir.



Astuce universelle

Placez toujours votre balise `<script src="...">` juste avant `</body>`, ou utilisez l'attribut `defer` dans le `<head>`. Cela garantit que le HTML est complètement chargé avant que JavaScript n'essaie de le modifier.

9.3 Variables et types de données

Les variables permettent de stocker des informations. Utilisez `let` et `const` (oubliez l'ancien `var`).

```
// const : la valeur ne changera jamais
const nom = 'Jean';
const anneeDeNaissance = 2001;

// let : la valeur peut être modifiée plus tard
let age = 25;
age = 26; // Mise à jour de l'âge
```

Type	Exemple	Description
String (Chaîne)	"Texte", 'Bonjour'	Du texte, entre guillemets ou apostrophes.

Number (Nombre)	42, 3.14	Entier ou décimal (sans guillemets).
Boolean (Booléen)	true, false	Valeur logique : vrai ou faux.
Array (Tableau)	['Pomme', 'Poire']	Une liste de plusieurs valeurs.
Object (Objet)	{ nom: 'Dupont', age: 25 }	Structure complexe avec propriétés (clés/valeurs).

9.4 Conditions et logique

```
let age = 25;

if (age >= 18) {
  console.log('Vous êtes majeur.');
```

```
} else if (age === 17) {
  console.log('Presque majeur !');
```

```
} else {
  console.log('Vous êtes mineur.');
```

```
}
```

⚠ Attention

En JavaScript, utilisez toujours `===` pour vérifier une égalité (vérifie la valeur ET le type). Évitez `==` qui peut causer des bugs : `0 == "0"` est vrai, mais `0 === "0"` est faux.

9.5 Boucles : répéter des actions

Les boucles permettent de répéter un bloc de code tant qu'une condition est vraie, ou pour parcourir une liste de valeurs.

```
// for – quand on connaît le nombre d'itérations
for (let i = 0; i < 5; i++) {
  console.log('Tour n°' + i); // Affiche Tour n°0 jusqu'à Tour n°4
}
```

```
// while – quand la condition est inconnue à l'avance
let compteur = 0;
while (compteur < 3) {
  console.log('Valeur : ' + compteur);
  compteur++;
}
```

```
// for...of – parcourir un tableau (moderne et lisible)
const fruits = ['Pomme', 'Banane', 'Cerise'];
for (const fruit of fruits) {
  console.log(fruit); // Pomme, Banane, Cerise
}
```

```
// for...in – parcourir les clés d'un objet
const personne = { nom: 'Dupont', age: 30, ville: 'Bruxelles' };
for (const cle in personne) {
```

```
console.log(cle + ' : ' + personne[cle]);
}
```

⚠ Attention — Boucle infinie

Assurez-vous que la condition de votre boucle while devient fausse à un moment. Oublier d'incrémenter le compteur (compteur++) bloque le navigateur indéfiniment.

9.6 Les tableaux et leurs méthodes

Les tableaux (arrays) stockent des listes de valeurs. JavaScript offre des méthodes puissantes pour les manipuler sans écrire de boucle manuellement.

```
const notes = [12, 18, 9, 15, 11];

// Accéder à un élément (index commence à 0)
console.log(notes[0]); // 12

// Ajouter / retirer
notes.push(17);        // Ajoute 17 en fin de tableau
notes.pop();           // Retire le dernier élément
notes.unshift(20);     // Ajoute 20 au début
notes.shift();         // Retire le premier élément

// Longueur du tableau
console.log(notes.length); // Nombre d'éléments

// forEach - parcourir (remplace for...of dans un style fonctionnel)
notes.forEach(note => console.log(note));

// map - transformer chaque élément (crée un NOUVEAU tableau)
const notesSur20 = notes.map(n => n + ' /20');
// ['12 /20', '18 /20', '9 /20', '15 /20', '11 /20']

// filter - garder seulement les éléments qui passent un test
const reussies = notes.filter(n => n >= 10);
// [12, 18, 15, 11]

// find - retourne le PREMIER élément qui correspond
const premiereNote18 = notes.find(n => n === 18); // 18

// includes - vérifier si une valeur est présente
console.log(notes.includes(18)); // true

// join - transformer en chaîne de caractères
console.log(notes.join(', ')); // "12, 18, 9, 15, 11"
```

Méthode	Effet	Retourne
push(val)	Ajoute val à la fin	Nouvelle longueur
pop()	Retire le dernier élément	Élément retiré

unshift(val)	Ajoute val au début	Nouvelle longueur
shift()	Retire le premier élément	Élément retiré
map(fn)	Transforme chaque élément via fn	Nouveau tableau
filter(fn)	Garde les éléments où fn est true	Nouveau tableau
find(fn)	Premier élément où fn est true	L'élément ou undefined
forEach(fn)	Exécute fn sur chaque élément	undefined
includes(val)	Vérifie la présence de val	true / false
join(sép)	Convertit en chaîne avec séparateur	String

9.7 Fonctions : créer des blocs réutilisables

```
// Déclaration classique
function direBonjour(prenom) {
  return 'Bonjour ' + prenom + ' !';
}
let message = direBonjour('Jean');
console.log(message); // Affiche : Bonjour Jean !

// Fonction fléchée (Arrow function) - syntaxe moderne
const additionner = (a, b) => {
  return a + b;
};

// Version condensée (1 seule expression)
const multiplier = (a, b) => a * b;
```

9.8 Le DOM : manipuler le HTML avec JavaScript

Le **DOM (Document Object Model)** est la représentation de votre page HTML que JavaScript peut lire et modifier.

```
// Sélectionner un élément
const titre = document.querySelector('h1');

// Modifier son texte
titre.textContent = 'Titre modifié par JavaScript';

// Modifier son style CSS
titre.style.color = '#2874A6';

// Ajouter une classe CSS
titre.classList.add('titre-important');

// Créer et insérer un nouvel élément
const nouveauPara = document.createElement('p');
nouveauPara.textContent = 'Paragraphe créé dynamiquement.';
document.body.appendChild(nouveauPara);
```

9.9 Les événements : réagir aux actions de l'utilisateur

Les événements sont au cœur de l'interactivité : clics, survols, frappes au clavier, soumission de formulaires...

```
const monBouton = document.querySelector('button');

// Écouter l'événement "click"
monBouton.addEventListener('click', () => {
  alert('Clic détecté !');
});

// Autres événements courants :
// 'mouseover' → survol avec la souris
// 'keydown' → touche enfoncée
// 'submit' → soumission de formulaire
// 'change' → valeur d'un champ modifiée
// 'load' → page entièrement chargée
```

9.10 Mini-Projet : Mode Sombre (Dark Mode)

Exemple concret liant HTML, CSS et JavaScript.

Le HTML

```
<button id="btn-theme">Passer en Mode Sombre</button>
<p>Un texte d'exemple sur notre page.</p>
```

Le CSS

```
/* Style de base (mode clair) */
body {
  background-color: white;
  color: black;
  transition: background-color 0.3s ease;
}

/* Classe ajoutée par le JavaScript */
.dark-mode {
  background-color: #1C2833;
  color: white;
}
```

Le JavaScript

```
const bouton = document.querySelector('#btn-theme');
const corpsPage = document.querySelector('body');
```

```
bouton.addEventListener('click', () => {  
  
  // toggle() ajoute la classe si absente, la retire si présente  
  corpsPage.classList.toggle('dark-mode');  
  
  // Adapter le texte du bouton  
  if (corpsPage.classList.contains('dark-mode')) {  
    bouton.textContent = 'Passer en Mode Clair';  
  } else {  
    bouton.textContent = 'Passer en Mode Sombre';  
  }  
});
```

9.11 Checklist des bonnes pratiques JavaScript

- ✓ Code clair : commentez votre code avec // pour expliquer la logique.
- ✓ Fichier JS séparé : gardez votre HTML propre avec un fichier .js externe.
- ✓ Tests console : utilisez console.log() pour vérifier vos variables.
- ✓ Nommage pertinent : let prixTotal au lieu de let p.
- ✓ Utilisez const par défaut, let si la valeur doit changer, jamais var.
- ✓ Vérifiez les erreurs dans la console DevTools (F12).